

## Data Wrangling

with pandas  
Cheat Sheet

http://pandas.pydata.org

## Syntax – Creating DataFrames

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 8 | 9 | 11 |
| 3 | 6 | 8 | 12 |

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = [1, 2, 3])
```

Specify values for each column.

```
df = pd.DataFrame(
    [14, 7, 10],
    [5, 8, 11],
    [6, 9, 12]),
    index=[1, 2, 3],
    columns=['a', 'b', 'c'])
```

Specify values for each row.

|   | a | b | c  |
|---|---|---|----|
| 1 | 4 | 7 | 10 |
| 2 | 8 | 9 | 11 |
| 3 | 6 | 8 | 12 |

```
df = pd.DataFrame(
    {"a": [4, 5, 6],
     "b": [7, 8, 9],
     "c": [10, 11, 12]},
    index = pd.MultiIndex.from_tuples(
        [['1', 'a'], ['2', 'b'], ['3', 'c']],
        names=['a', 'b'])
```

Create DataFrame with a MultiIndex

## Method Chaining

Most pandas methods return a DataFrame so that another pandas method can be applied to the result. This improves readability of code.

```
df = (pd.melt(df)
      .rename(columns={
          "variable": "var",
          "value": "val"})
      .query('val > 200'))
```

## Tidy Data – A foundation for wrangling in pandas

In a tidy data set:

Each variable is saved in its own column

Each observation is saved in its own row

Tidy data complements pandas's vectorized operations. pandas will automatically preserve observations as you manipulate variables. No other format works as intuitively with pandas.

$M \times A$

## Reshaping Data – Change the layout of a data set

pd.melt(df)

Gather columns into rows.

df.pivot(columns='var', values='val')

Spread rows into columns.

pd.concat([df1, df2])

Append rows of DataFrames

pd.concat([df1, df2], axis=1)

Append columns of DataFrames

```
df.sort_values('mpg')
Order rows by values of a column (low to high).

df.sort_values('mpg', ascending=False)
Order rows by values of a column (high to low).

df.rename(columns = {'y': 'year'})
Rename the columns of a DataFrame

df.reset_index()
Sort the index of a DataFrame

df.reset_index()
Reset index of DataFrame to row numbers, moving index to columns.

df.drop(columns=['Length', 'Height'])
Drop columns from DataFrame
```

## Subset Observations (Rows)

```
df[df.length > 7]
Extract rows that meet logical criteria.

df.drop_duplicates()
Remove duplicate rows (only considers columns).

df.head(n)
Select first n rows.

df.tail(n)
Select last n rows.
```

```
df.sample(frac=0.5)
Randomly select fraction of rows.

df.sample(n=10)
Randomly select n rows.

df.iloc[10:20]
Select rows by position.

df.nlargest(n, 'value')
Select and order top n entries.

df.nsmallest(n, 'value')
Select and order bottom n entries.
```

## Subset Variables (Columns)

```
df[['width', 'length', 'species']]
Select multiple columns with specific names.

df['width'] or df.width
Select single column with specific name.

df.filter(regex='reg-exp')
Select columns whose name matches regular expression regex.
```

| regex (Regular Expressions) Examples |                                                              |
|--------------------------------------|--------------------------------------------------------------|
| "^L"                                 | Matches strings containing a period '.'                      |
| "length\$"                           | Matches strings ending with word 'length'                    |
| "^year"                              | Matches strings beginning with the word 'year'               |
| "^L[1-5]"                            | Matches strings beginning with 'L' and ending with 1,2,3,4,5 |
| "^[^Season]"                         | Matches strings except the string 'Season'                   |

```
df.loc[:, 'x2': 'x4']
Select all columns between x2 and x4 (inclusive).

df.iloc[:, 1:2, 5]
Select columns in positions 1, 2 and 5 (first column is 0).

df.loc[df['a'] > 10, ['a', 'c']]
Select rows meeting logical condition, and only the specific columns.
```

Figure 2: Pandas cheat sheet (Reference: [https://pandas.pydata.org/Pandas\\_Cheat\\_Sheet.pdf](https://pandas.pydata.org/Pandas_Cheat_Sheet.pdf))

Let's first read a configuration file using the Python script.  
A configuration file looks like this:

```
1. [FileLocations]
2. master.filename=project-data.xlsx
3. auditstatus.filename=audit-status.xlsx
```

To read this file, use the following script:

```
1. import pandas as pd
2. import numpy as np
3. import configparser as cfg
4. import array as arr
5. from pandas import DataFrame
6. import matplotlib.pyplot as plt
7. import io
8.
9. #Read Properties file
10. config = cfg.RawConfigParser()
11. config.read('master.properties')
```

```
12. #access data from properties file
13. print(config.get('FileLocations', 'master.filename'))
```

Let's now read data from the Excel file, the name for which is available in the *properties* file:

```
1. #Read Data
2. projectData = pd.read_excel(config.get('FileLocations',
    'master.filename'))
3. auditStatus = pd.read_excel(config.get('FileLocations',
    'auditstatus.filename'))
4. #Print data on console
5. print("=====")
6. print(projectData)
7. print("=====")
8. print(auditStatus)
```

Figure 3 gives the output of two DataFrames.